```
#_____
_____

#                                                    Date : 08-04-2026
from threading import *
import time

# currentthread() -- single thread

'''
def func():
print("func()", current_thread().name)
for i in range(5):
print("Yo")
def display():
print("display", current_thread())
for i in range(5):
print("see you")
func()
display()
'''

# multithread
'''
def func():
print("func()",current_thread().name)
for i in range(5):
print("Yo")
def display():
print("display()",current_thread().name)
for i in range(5):
print("see you")

t1=Thread(target=func,name="aizen")
t2=Thread(target=display,name="urahara")
t1.start()
t2.start()
'''

# possible to pass arguments to the thread
'''
def func(n):
print("func()",current_thread().name)
for i in range(5):
print(n)
def display(m):
print("display()",current_thread().name)
for i in range(5):
print(m)
t1=Thread(target=func,args=("Yo",))
t2=Thread(target=display,args=("see you",))
t1.start()
t2.start()
'''
```

```
#keword arguments
'''
def func(m,n):
print("func()",current_thread().name)
for i in range(5):
print(m+n)
def display(x,y):
print("display()",current_thread().name)
for i in range(5):
print(x+y)
t1=Thread(target=func,kwargs={"m": "Yo","n": "koso"})
t2=Thread(target=display,kwargs={"x": "see ","y": "you"})
t1.start()
t2.start()
'''
#
'''
def func():
for i in range(5):
print("Yo")
def display():
for i in range(5):
print("see you")
s=time.time()
func()
display()
e=time.time()
print("Time taken",e-s)
'''

#
'''
def func():
for i in range(5):
print("Yo")
def display():
for i in range(5):
print("see you")

t1=Thread(target=func)
t2=Thread(target=display)

s=time.time()
t1.start()
t2.start()
t1.join()
t2.join()
e=time.time()
print("Time taken",e-s)
'''

# class threading
'''
```

```
class A:
def func(self):
for i in range (5):
print("Yo")
def display(self):
for i in range(5):
print("see you")
a=A()
t1=Thread(target=a.func)
t2=Thread(target=a.display)
t1.start()
t2.start()
'''
# arbitrary positional arguments in class threading

'''
class A:
def func(self,a):
for i in range (5):
print(a)
def display(self,m):
for i in range(5):
print(m)
a=A()
t1=Thread(target=a.func,args=("Yo",))
t2=Thread(target=a.display,args=("see you",))
t1.start()
t2.start()
'''

# Arbitary keyword arguments
'''
class A:
def func(self,a,b):
for i in range (5):
print(a+b)
def display(self,m,n):
for i in range(5):
print(m+n)
a=A()
t1=Thread(target=a.func,kwargs={"a": "Yo","b": "koso"})
t2=Thread(target=a.display,kwargs={"m": "see ","n": "you"})
t1.start()
t2.start()
'''
#_____
_____

#                                                    Date : 09-04-2026
# inheriting Thread class so no need for target and args
'''
class A(Thread):
print("A class",current_thread().name)
def run(self):
```

```
for i in range (5):
print("Bankai")
class B(Thread):
print("B class",current_thread().name)
def run(self):
for i in range(5):
print("Shikai")
a=A()
b=B()
a.run()  # normal function call so it will run in main thread
b.run()
'''
# to run in separate thread we need to call start() method
'''
class A(Thread):

def run(self):
print("A class",current_thread().name)
for i in range(5):
print("Bankai")
class B(Thread):
def run(self):
print("B class",current_thread().name)
for i in range(5):
print("Shikai")
a=A()
a.start()
b=B()
b.start()
'''

# time sleep() method
'''
def func():
for i in range (5):
print("Bankai")
time.sleep(1)
def display():
for i in range(5):
print("Shikai")
func()
display()
'''
# time sleep() method in multithreading
'''
def func():
for i in range (5):
print("Bankai")
time.sleep(1)
def display():
for i in range(5):
print("Shikai")
time.sleep(1)
t1=Thread(target=func)
```

```
t2=Thread(target=display)
t1.start()
t2.start()
'''

# init thread
'''
class A(Thread):
def __init__(self,m):
Thread.__init__(self)
self.m=m
def run(self):
for i in range(5):
print(self.m)

class B(Thread):
def __init__(self,n):
Thread.__init__(self)
self.n=n
def run(self):
for i in range(5):
print(self.n)
a=A("Bankai")     # here order is important because we are passing argument to the constructor
and it will be used in run method
a.start()
b=B("Shikai")
b.start()
'''

# join() method
'''
def func():
for i in range(5):
print("shikkai")
def show():
for i in range(5):
print("bankai")
def display():
for i in range(5):
print("kyoka suigetsu")
t1=Thread(target=func)
t2=Thread(target=show)
t3=Thread(target=display)
t1.start()
t2.start()
t1.join()  # it will wait for t1 to complete and then it will start t3
t3.start()

'''

# join() method with timeout
'''
def func():
for i in range(5):
```

```
print("shikkai")
def show():
for i in range(5):
print("bankai")
def display():
for i in range(5):
print("kyoka suigetsu")
t1=Thread(target=func)
t2=Thread(target=show)
t3=Thread(target=display)
t1.start()
t2.start()
t2.join(1)  # it will wait for t2 to complete for 1 second and then it will start t3
t3.start()

'''

# time taken by join() method with timeout
'''
def func():
for i in range(5):
print("shikkai")
time.sleep(1)
def show():
for i in range(5):
print("bankai")
def display():
for i in range(5):
print("kyoka suigetsu")
t1=Thread(target=func)
t2=Thread(target=show)
t3=Thread(target=display)
t1.start()
t2.start()
t2.join(3)  # it will wait for t2 to complete for 3 seconds and then it will start t3
t3.start()
'''
#
'''
def func():
time.sleep(1)
t3.join()  # it will wait for t3 to complete and then it will start t1
for i in range(5):
print("shikkai")

def show():
for i in range(5):
print("bankai")
def display():
for i in range(5):
print("kyoka suigetsu")
t1=Thread(target=func)
t2=Thread(target=show)
t3=Thread(target=display)
```

```
t1.start()
t2.start()
t3.start()
'''

#
'''
def func():
for i in range(5):
print("shikkai")
def show():
for i in range(5):
print("bankai")
def display():
for i in range(5):
print("kyoka suigetsu")
t1=Thread(target=func)
t2=Thread(target=show)
t3=Thread(target=display)
t1.start()
t1.join()  # it will wait for t1 to complete and then it will start t2
t2.start()
t2.join()  # it will wait for t2 to complete and then it will start t3
t3.start()
'''

# Thread name we can set thread name by using name parameter in Thread() constructor or by
using setName() method

'''
def func():
print("func()",current_thread().name)
for i in range(5):
print("shikkai")
def display():
print("display()",current_thread().name)
for i in range(5):
print("bankai")
t1=Thread(target=func)
t1.name="aizen"
t2=Thread(target=display)
t2.name="urahara"

t1.start()
t2.start()
'''

#_____
_____

#                                                    Date : 10-04-2026

# is_alive() method
'''
```

```
def func():
for i in range(5):
print("shikkai")
def show():
for i in range(5):
print("bankai")

t1=Thread(target=func)
t2=Thread(target=show)
print("Before t1",t1.is_alive())  # it will return False because t1 is not started yet
print("Before t2",t2.is_alive())  # it will return False because t2 is not started yet

t1.start()
t2.start()
print("After t1",t1.is_alive())  # it will return True because t1 is started
print("After t2",t2.is_alive())  # it will return True because t2 is started
'''
# active_count() method returns the number of active threads in the current thread's thread group. It includes the main thread and all the threads that are currently running.
#print("Active threads",active_count())  # it will return 3 because main thread, t1 and t2 are active

# active_count() method with time sleep() method
# main thread will be active until all the threads are completed
'''
def func():
for i in range(5):
print("shikkai")
time.sleep(1)
def show():
for i in range(5):
print("bankai")

t1=Thread(target=func)
t2=Thread(target=show)

print("Before t1",t1.is_alive())
print("Before t2",t2.is_alive())

t1.start()
t2.start()

print("Active threads",active_count())
'''

# enumerate() method returns a list of all the active threads in the current thread's thread group. It includes the main thread and all the threads that are currently running.
'''
def func():
for i in range(5):
print("shikkai")
time.sleep(1)
def show():
for i in range(5):
print("bankai")
```

```
time.sleep(1)

t1=Thread(target=func)
t2=Thread(target=show)
print("Before t1",t1.is_alive())
print("Before t2",t2.is_alive())
t1.start()
t2.start()
print("Active threads",active_count())
print("Enumerate threads",enumerate())  # it will return a list of all the active threads in the
current thread's thread group
'''

# process ID -- os.getpid()
'''
import os
from threading import *

def func():
print("process ID of func",os.getpid())
print("shikkai")
def display():
print("process ID of display",os.getpid())
print("bankai")

t1=Thread(target=func)
t2=Thread(target=display)
t1.start()
t2.start()
print("process ID of main thread",os.getpid())  # it will return the process ID of the main thread
'''



# thread ID -- t1.ident
'''
import os
from threading import *

def func():
print("thread ID of func",current_thread().ident)
print("shikkai")
def display():
print("thread ID of display",current_thread().ident)
print("bankai")
t1=Thread(target=func)
t2=Thread(target=display)
t1.start()
t2.start()
print("thread ID of main thread",current_thread().ident)  # it will return the thread ID of the main
thread
'''

# daemon thread -- a thread that runs in the background and does not block the main thread from
exiting. It is used for tasks that are not critical to the main thread and can be safely ignored if the
```

main thread exits.

Daemon threads are running background
Daemon threads are used to provide the supports for non daemon thread
Daemon threads are running continuously in memory
Garbage collecter is the best example of daemon thread
Garbage collector is used to delete useless objects from the memory at the time of program execution
By using daemon property we can set a thread as daemon thread, as well as by using this property we can check s thread is daemon thread or not
Remember main thread is always non-deamon and we can't change main thread as daemon thread
Once a thread started then we can't change its nature , so main thread started by pvm , so its not in our hand, that's why we can't change the default nature of main thread.
Except main thread, remaining all threads nature depends nature depends on its parent, if the parent is daemon, then child also daemon , and vice-versa.
we can change the nature of other thread except main thread.
Daemon thread are widely used to maintain log record, grammer checker, scrap data from web in the background.

'''
from threading import *
import time
def func():
print("func()",current_thread().daemon)
for i in range(5):
print("shikkai")
def display():
print("display()",current_thread().daemon)
for i in range(5):
print("bankai")
t1=Thread(target=func)
t2=Thread(target=display)
t1.start()
t2.start()
'''

daemon thread with time sleep() method -- if the main thread exits then the daemon thread will also exit even if it is not completed
in real time like bus ticket booking website, we can use daemon thread to maintain log record, if the main thread exits then the daemon thread will also exit even if it is not completed, so it will save the memory and also it will not create any problem because log record is not critical to the main thread.
'''
#
from threading import *
import time
def func():
print("func()",current_thread().daemon)
for i in range(5):
time.sleep(0.5)
print("shikkai")

```
t1=Thread(target=func,daemon=True)
t1.start()
time.sleep(1)
print("main thread is exiting")  # main thread is exiting after 1 second, so the daemon thread will also exit even if it is not completed
'''
#
'''
from threading import *
import time
def func():
print("func()",current_thread().daemon)
for i in range(5):
time.sleep(0.5)
print("shikkai")

def display():
print("display()",current_thread().daemon)
for i in range(5):
time.sleep(1)
print("bankai")

t1=Thread(target=func)
t2=Thread(target=display,daemon=True)
t1.start()
t2.start()
'''



'''
from threading import *
import time
def func():
print("func()",current_thread().daemon)
for i in range(5):
print("shikkai")
def display():
print("display()",current_thread().daemon)
for i in range(5):
print("bankai")
time.sleep(1)
t1=Thread(target=func)
t2=Thread(target=display)
t2.daemon=True  # it will make t2 a daemon thread
# t2.setDaemon(True)  # it will also make t2 a daemon thread
# t2=Thread(target=diplay,daemon=True) # it will alo make t2 a daemon thread)
t1.start()
t2.start()
'''
# daemon thread inside a normal thread -- it will automatically crete daemon thread because its parent is daemon thread
'''
from threading import *
```

```
import time
def func():
print("func()",current_thread().daemon)
for i in range(5):
print("shikkai")
t2=Thread(target=display)
t2.start()
print("t2 is daemon",t2.daemon)
def display():
print("display()",current_thread().daemon)
for i in range(5):
time.sleep(1)
print("bankai")
t1=Thread(target=func,daemon=True)
t1.start()
time.sleep(2)
print("main thread is exiting")
'''
# daemon thread inside a non daemon thread -- using daemon=False but its parent is daemon
thread
'''
from threading import *
import time

def func():
print("func()",current_thread().daemon)
for i in range(5):
print("shikkai")
t2=Thread(target=display,daemon=False)
t2.start()

def display():
print("display()",current_thread().daemon)
for i in range(5):
print("bankai")
time.sleep(1)

t1=Thread(target=func,daemon=True)
t1.start()
time.sleep(2)
print("main thread is exiting")
'''

# thread inside a daemon thread -- using daemon=True but its parent is daemon thread
'''
def func():
print("func()",current_thread().daemon)
for i in range(5):
print("shikkai")
t2=Thread(target=display,daemon=True)  # it will return the current thread object
t2.start()
def display():
print("display()",current_thread().daemon)
for i in range(5):
```

```
time.sleep(1)
print("bankai")
t1=Thread(target=func)
t1.start()
time.sleep(1)
print("main thread is exiting")
'''
```

#_____
_____

Date : 11-04-2026
saturday --
#_____
_____

Date : 12-04-2026
sunday -- holiday

#_____
_____

Monday Date : 13-04-2026

```
# Tradditional method for daemon -- is depriciated
'''
from threading import *
import time

def func():
for i in range(5):
print("yokoso")
time.sleep(1)


def display():
for i in range(5):
print("saakasam no sekai")
time.sleep(.5)
t1=Thread(target=func)
t1.setDaemon(True)
t2=Thread(target=display)

t1.start()
t2.start()
'''
#----------
```

GIL -- Global Interpretor Lock so we use from threading import * to use multithreading

Race condition

here we dont use race condition so ticket possibly booked bu both thread
'''
from threading import *

avl_seat=2

def booking(seat):
global avl_seat
if seat<=avl_seat:
print(f"bookking successfully {seat} by {current_thread().name}")
avl_seat-=seat
print(f"{avl_seat} are available")
elif seat>avl_seat:
print(f"only {avl_seat} but you have booked {seat}")
print(f"{avl_seat} are available")

t1=Thread(target=booking,args=(2,),name="Aizen")
t2=Thread(target=booking,args=(2,),name="Urahara")
t1.start()
t2.start()

'''

Thread syncronization

simple lock
rlock
semaphore

Locked

Once a thread obtains the lock, It goes into the locked state
l.acquire() method is used to lock a thread

Unlocked

once a thread releses the lock, It goes into unlocked state
l.release() method is used to unlock a thread

using l.acquire() and l.release()
'''
from threading import *

avl_seat=2
l=Lock()

def booking(seat):
global avl_seat
l.acquire()
if seat<=avl_seat:

```
print(f"bookking successfully {seat} by {current_thread().name}")
avl_seat-=seat
print(f"{avl_seat} are available")
elif seat>avl_seat:
print(f"only {avl_seat} but you have booked {seat}")
print(f"{avl_seat} are available")
l.release()
t1=Thread(target=booking,args=(2,),name="Aizen")
t2=Thread(target=booking,args=(2,),name="Urahara")
t1.start()
t2.start()
'''

# with l

'''
from threading import *

avl_seat=2
l=Lock()
def booking(seat):
global avl_seat
with l: # it is also work like above l.acquire and l.release()
if seat<=avl_seat:
print(f"booking successfully {seat} by {current_thread().name}")
avl_seat-=seat
print(f"{avl_seat} are available")
elif seat>avl_seat:
print(f"only {avl_seat} but you have booked {seat}")
print(f"{avl_seat} are available")
t1=Thread(target=booking,args=(2,),name="Aizen")
t2=Thread(target=booking,args=(2,),name="Urahara")
t1.start()
t2.start()
'''

# Multi processing

# is a python technique where multiple processes run in parallel.

## Mlti Threading              --  Multi Processing

## same memories              --  Each run with its own memory
## multi Threading is IO bound    --  Multiprocessing is cpu bound
## GIL is affecting           --  GIL is not affecting

#

'''
import multiprocessing

def func():
for i in range(5):
print("Yokoso")
```

```
def display():
for i in range(5):
print("Sakasama no sekai")

if __name__=="__main__":
p1=multiprocessing.Process(target=func)
p2=multiprocessing.Process(target=display)    # if we using this line we have to import
multiprocessing
p1.start()
p2.start()
'''
#
'''
from multiprocessing import *
import time

def func():
print(f"start processing {current_process().name}")
time.sleep(2)
print(f"end processing {current_process().name}")

if __name__ == "__main__" :
p1=Process(target=func,name="Nel")
p2=Process(target=func,name="ichigo")

p1.start()
p2.start()
'''
#  Process() with join and without also
'''
from multiprocessing import *
import time

def func(n):
print(f"{n} start processing {current_process().name}")
time.sleep(2)
print(f"{n} end processing {current_process().name}")

if __name__ == "__main__" :
p1=Process(target=func,name="Nel",args=[1])
p2=Process(target=func,name="ichigo",args=[2])

p1.start()
p1.join()
p2.start()

'''

# with and pool -- multiprocessing if we did  more inputs (like 10000000) it will be faster
'''
from multiprocessing import *
import os
import time
```

```
def func(n):
time.sleep(1)
print(f"{os.getpid()} it runs to {n}")
return n**2
if __name__=="__main__":
num=[1,2,3,4,5]
with Pool(processes=2) as p:   # here it automatically create processes pool(processes=x?)
res=p.map(func,num)
print("answer =",res)
'''

#_____
_____

#                                                    Date : 14-04-2026

from multiprocessing import *
import os
import time
import math

# math.fctorial
'''
def func(n):
print(f"{n} factorial of {math.factorial(n)} in {os.getpid()} ")

if __name__ == "__main__":
p1=Process(target=func,args=[5])
p2=Process(target=func,args=[4])
p1.start()
p2.start()
'''
#
'''
def func():
for i in range(5):
print("Onepiece")
def display():
for i in range(5):
print("DragonBall")
if __name__=="__main__":
p1=Process(target=func)
p2=Process(target=display)
p1.start()
# p1.join()
p2.start()
'''
# factorial with for loop and for loop join
'''
def func(n):
print(f"{n} factorial of {math.factorial(n)} in {os.getpid()}")

if __name__=="__main__":
```

```
num=[5,4,3,6]
pro=[]
for i in num:
p=Process(target=func,args=(i,))
pro.append(p)
p.start()
for j in pro:
j.join()  # it will wait for all the processes to complete and then it will print the answer
print("All processes are completed")  # it will print after all the processes are completed
'''
# same func but with pool method
'''
def func(n):
time.sleep(1)
print(f"{n} factorial of {math.factorial(n)} in {os.getpid()}")

if __name__=="__main__":

num=[5,4,7,8,3,2,6,12,13, 14,15,16]
with Pool(processes=4) as p: # 4 process only
res=p.map(func,num)
'''


#  q=Queue() -- x.put(i) and x.get()
'''
def put_data(m):
a=[4,5,6,7,8]
for i in a:
m.put(i)  # putting data one by one  in q=Queue()
def get_data(n):
while not n.empty():
print("get data",n.get()) # one by one gettting
if __name__=="__main__":
q=Queue()
p1=Process(target=put_data,args=(q,))
p2=Process(target=get_data,args=(q,))
p1.start()
p2.start()
'''
#
'''
def put_data(m):
a=[4,5,6,7,8]
for i in a:
m.put(i)  # putting data one by one  in q=Queue()
def get_data(n):
while not n.empty():
print("get data",n.get())
if __name__=="__main__":
q=Queue()
p1=Process(target=put_data,args=(q,))
p2=Process(target=get_data,args=(q,))
p1.start()
p1.join()          # but here one by one all are putting
```

```
p2.start()         # after one one by one we getting
p2.join()
print("All processes are completed") # last it will print
'''



# Tea Master
import random


# mormal
'''
def tea(n):
print(f"customer{n} order a tea and the order taken by {os.getpid()}")
time.sleep(random.randint(1,3))
print(f"order placed to {n} and prepared by {os.getpid()}")

if __name__=="__main__":
a=[1,2,3,4,5]
with Pool(processes=3) as p:
res=p.map(tea,a)
'''
# with queue
def get_tea(q):
while not q.empty():
a=q.get()
print(f"customer {a} ordered a tea. order taken by {os.getpid()}")
time.sleep(random.randint(1,3))
print(f"order placed to {a} and prepared by {os.getpid()}")


if __name__=="__main__":
a=[1,2,3,4,5]
q=Queue()
for i in a:
q.put(i)
p1=Process(target=get_tea,args=(q,))
p2=Process(target=get_tea,args=(q,))
p3=Process(target=get_tea,args=(q,))
p1.start()
p2.start()
p3.start()


# queue with pool not completed
'''
def put_tea(q):

print(f"customer{q.put()} order a tea and the order taken by {os.getpid()}")

def get_tea(q):
while not q.empty():
time.sleep(random.randint(1,3))
print(f"order placed to {q.get()} and prepared by {os.getpid()}")
```

```
if __name__=="__main__":
q=Queue()
a=[1,2,3,4,5]
for i in a:
q.put(i)

with Pool(processes=3) as p:
p.apply_async(put_tea,args=(q,))
p.apply_async(get_tea,args=(q,))
'''

#_____
_____
```